DT081A – Signal and Image Processing

Laboratory 1 Report

Sections 1.1–1.3, 2.1–2.3 and 3.1 – Signal and Image Processing

Jakob Obid

4 November 2026

1 Introduction

This part of Lab 1 compares sine, square and sawtooth waves. It shows how frequency and addition change a signal. FFTs were also used to study fixed frequencies, noise and a chirp. I also tested how different sampling rates affect a signal. I also compared image resampling and requantization. I also compared image quality using PSNR and SSIM. I also measured how Gaussian noise and filtering changed image PSNR. I also analyzed audio recordings and tested downsampling.

2 Experimental Setup

NumPy and Matplotlib were used. All signals use the teacher's time vector, `t = np.linspace(0, 1, 1000)`: 1,000 points from 0 to 1 second, including both ends. The starting signals are a 10 Hz sine and a 5 Hz square. Their amplitudes were kept unchanged. For Section 1.2, the original 10 Hz and 20 Hz sines have amplitudes 1 and 0.5. The FFT uses the time step between the given points. I used the teacher's sampling examples and tested a 23 Hz sine wave sampled at 8 Hz. Pillow loaded Lenna and a grass texture as grayscale images. PSNR used floating-point pixel errors, and SSIM used a data range of 255. Gaussian noise used seed 31415, and Gaussian filtering used spatial $\sigma = 1$. SciPy loaded the piano and thunderstorm WAV files; stereo channels were averaged before peak normalization. Spectrograms used 1,024-sample windows with 512-sample overlap; the frequency estimate used 1% of the strongest non-DC FFT magnitude.

3 Results and Analysis

3.1 Signal Generation and Visualization

Q1. Modify the code to create a sawtooth wave. Describe the mathematical function you used.

Answer: The function is $s(t) = 2(5t - \lfloor 5t \rfloor) - 1$. Here, $\lfloor 5t \rfloor$ means rounding down to a whole number. Subtracting it leaves the part of a cycle between 0 and 1. Scaling by 2 and subtracting 1 gives a rising ramp from -1 towards $+1$. It jumps back every 0.2 seconds, so its frequency is 5 Hz (Figure 1).

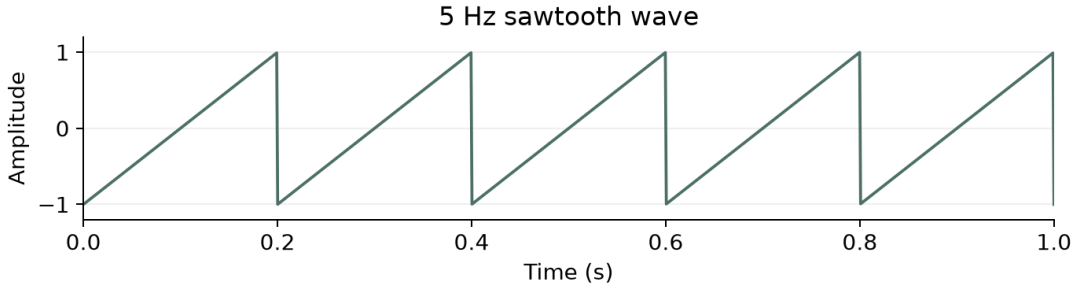


Figure 1: Five rising ramps during one second. Each ramp ends with a jump back.

Q2. Change the frequency of the sine wave to 20 Hz. How does this affect the plot?

Answer: The 20 Hz sine has 20 cycles in one second instead of 10. The waves are closer together in time. The amplitude stays the same (Figure 2).

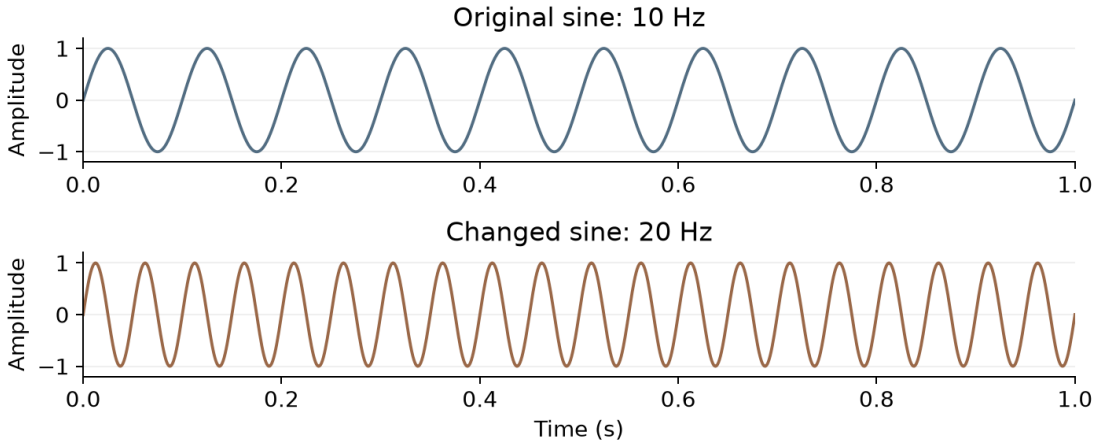


Figure 2: The same one-second interval and amplitude limits show twice as many cycles at 20 Hz.

Q3. What happens to the square wave if you change the sign function to `np.ceil(np.sin(2 * np.pi * 5 * t)) * 2 - 1`? Explain why.

Answer: The two expressions give the same value at 999 of the 1,000 samples. Only the first sample differs: at $t = 0$, `np.sign()` gives 0, while the replacement gives -1.

For a positive sine value, rounding up gives 1, so the replacement gives $+1$. For a negative value strictly between -1 and 0, rounding up gives 0, so the result is -1. At zero, it also gives -1, while `np.sign(0)` gives 0. If the sine is exactly -1, the replacement gives $2(-1) - 1 = -3$. No sampled sine value was exactly -1, so -3 did not appear in these results.

Q4. Create a composite signal by adding the sine wave and square wave together. Plot this new signal and describe its characteristics.

Answer: The original 10 Hz sine was added to the 5 Hz square. The sine gives the curved parts, and the square gives sudden jumps. The sampled values range from about -2 to $+2$ (Figure 3).

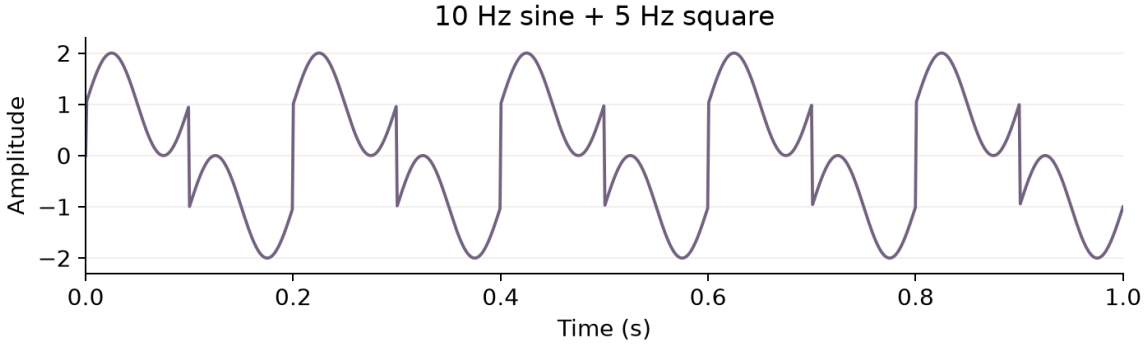


Figure 3: The original sine and square added together, without changing their amplitudes.

Q5. How might understanding different signal types be useful in real-world applications? Provide an example.

Answer: A square wave can act as a clock signal in a digital circuit. Its regular changes tell parts of the circuit when to act. Its frequency tells us how often these changes repeat.

3.2 Fourier Transform and Frequency Domain Analysis

Q1. What frequencies are present in the signal? How can you identify them from the frequency domain plot?

Answer: The signal contains 10 Hz and 20 Hz. They give the two largest FFT peaks, at 9.99 and 19.98 Hz (Figure 4). Using the actual time step gives FFT bins 0.999 Hz apart. The peaks are at the nearest bins to the input frequencies.

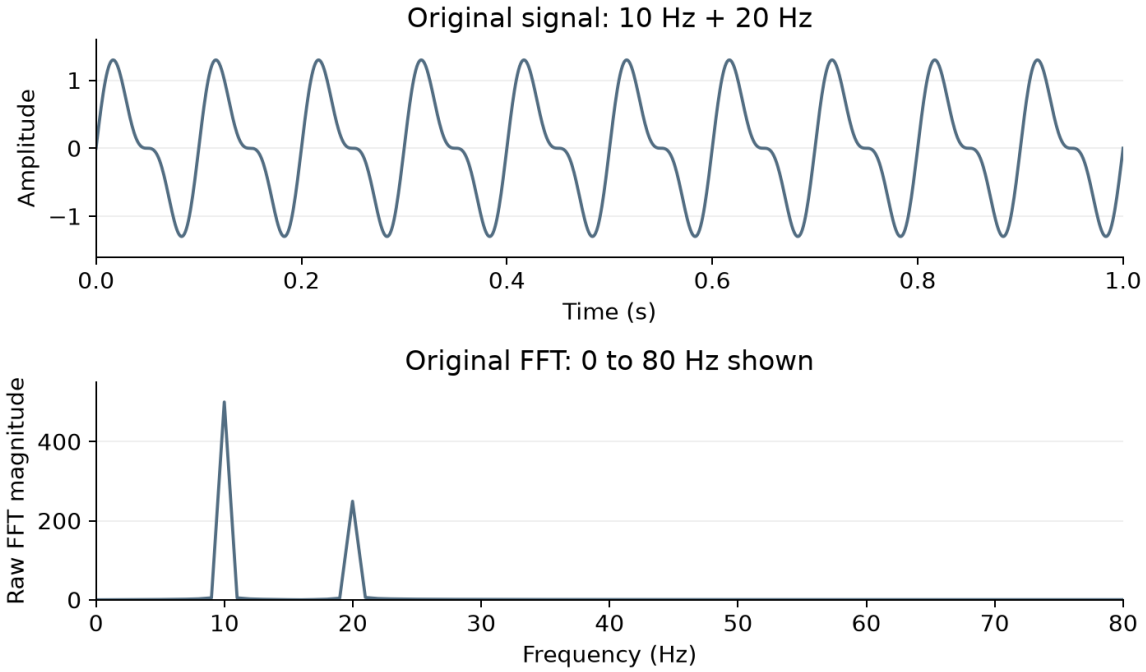


Figure 4: The original two-sine signal and its FFT magnitude. The frequency plot shows 0–80 Hz.

Q2. Modify the generate_signal function to include a 30 Hz component. How does this change the frequency domain representation?

Answer: A 30 Hz sine with amplitude 0.3 was added. A new peak appears at 29.97 Hz, with a raw FFT magnitude of 148.72. The 10 Hz and 20 Hz peaks remain (Figure 5).

Q3. What happens to the frequency domain plot if you add random noise to the signal? Implement this and explain your observations.

Answer: Gaussian noise with standard deviation 0.18 and seed 1729 was added to the original signal. Small, irregular values appear across the spectrum, while the two largest peaks stay at 9.99 and 19.98 Hz. This happens because the noise contains many frequencies (Figure 5).

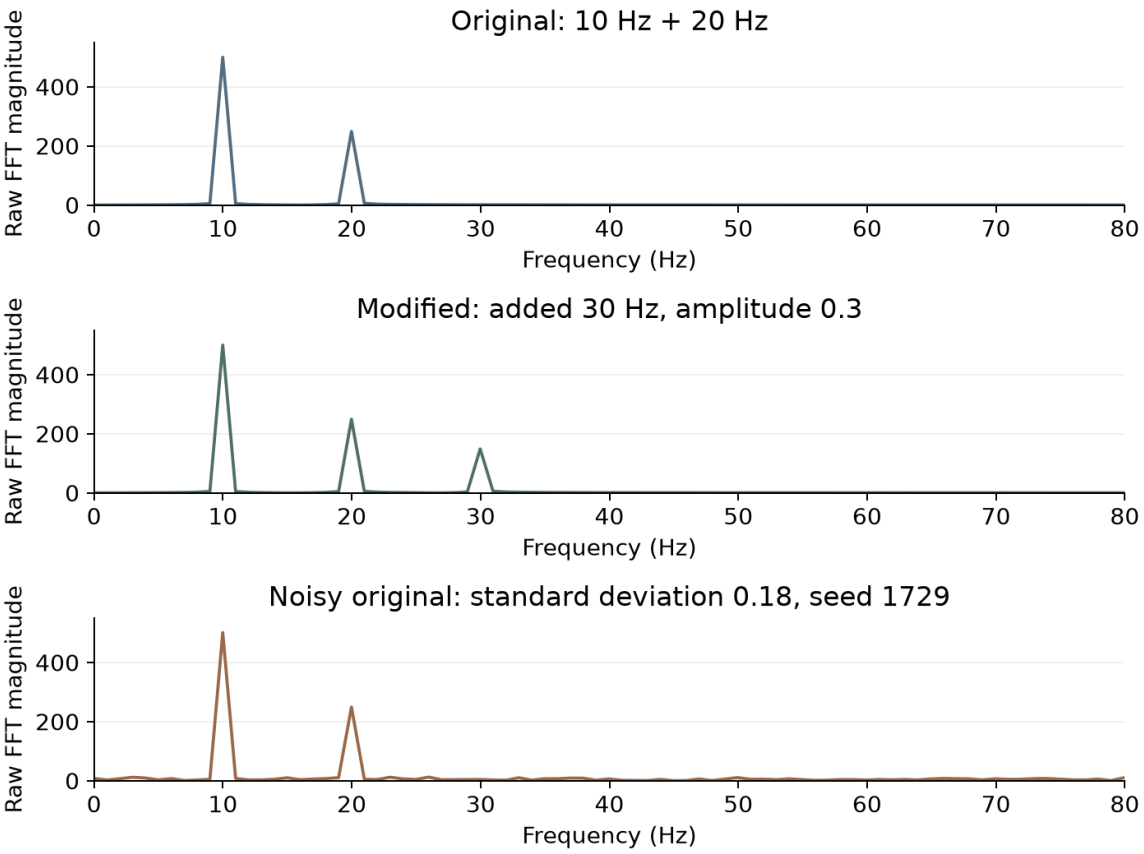


Figure 5: Original, modified and noisy spectra, using the same frequency and magnitude limits. Noise was added to the original signal.

Q4. Create a function that generates a signal with a frequency that changes over time (e.g., a chirp signal). Plot this signal in both time and frequency domains and explain what you observe.

Answer: The function sweeps from 15 Hz to 65 Hz in one second. The waves get closer together over time. The FFT shows a broad band around this range instead of two narrow peaks. It shows which frequencies are present, but not when each one occurs (Figure 6).

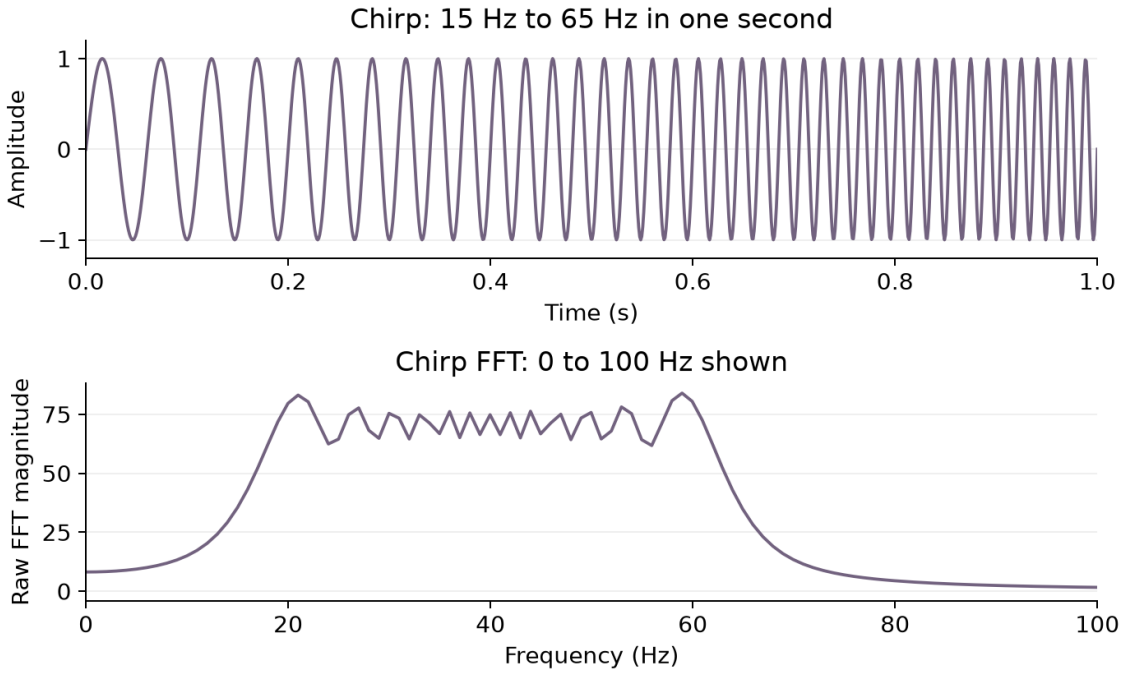


Figure 6: The chirp speeds up during the recording. Its FFT magnitude is shown over 0–100 Hz.

Q5. Describe a practical application where understanding the frequency domain of a signal is important.

Answer: A sensor can measure a machine's vibration. Its frequency peaks can be compared with a normal reading. New or much stronger peaks can help find a fault.

3.3 Sampling and Aliasing

Q1. What is the Nyquist rate for the given signal? Is the current sampling rate sufficient?

Answer: For a 10 Hz wave, the Nyquist rate is 20 Hz. The given 15 Hz sampling rate is too low, so the sampled signal shows a false 5 Hz pattern (Figure 7).

Q2. Increase the sampling rate to 25 Hz. How does this affect the sampled signal's representation of the original signal?

Answer: I increased the sampling rate to 25 Hz. This is above the 20 Hz Nyquist rate, so there is no aliasing. The extra sample points represent the original 10 Hz wave better (Figure 7).

Q3. Set the signal frequency to 20 Hz and the sampling rate to 30 Hz. Describe and explain the resulting aliasing effect.

Answer: I sampled a 20 Hz sine wave at 30 Hz. This is below the required 40 Hz Nyquist rate. The sampled points behave like a -10 Hz sine wave because the sampling rate is too low (Figure 7).

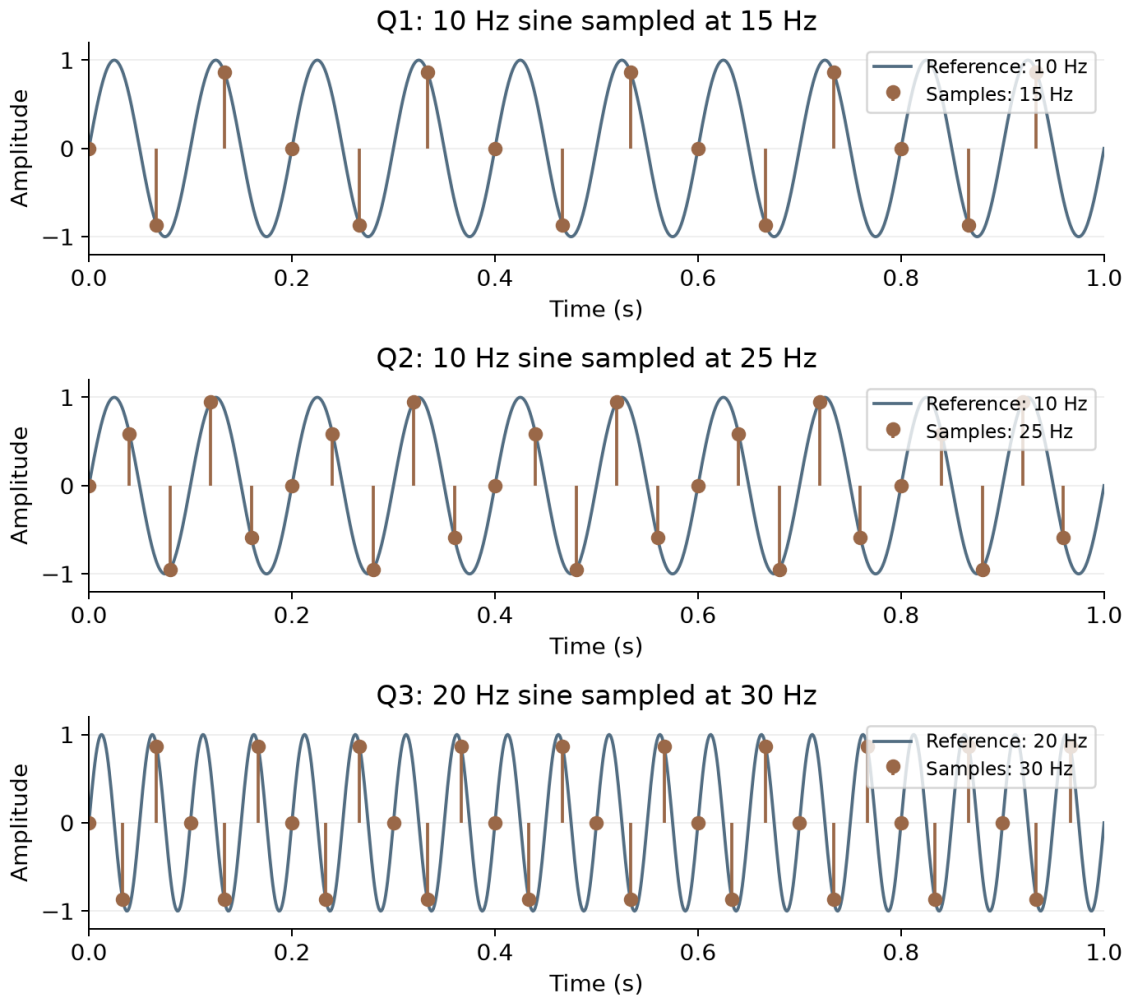


Figure 7: Sampling comparison for the three given cases. The 25 Hz sampling rate represents the 10 Hz signal without aliasing.

Q4. Create a function that demonstrates the "wagon-wheel effect" by sampling a high-frequency sinusoid at a much lower rate. Plot the original and sampled signals, and explain the visual effect.

Answer: I sampled a 23 Hz sine wave at 8 Hz. The sampled points match a 1 Hz wave with its sign reversed. This makes a fast repeating signal look much slower, like the wagon-wheel effect (Figure 8).

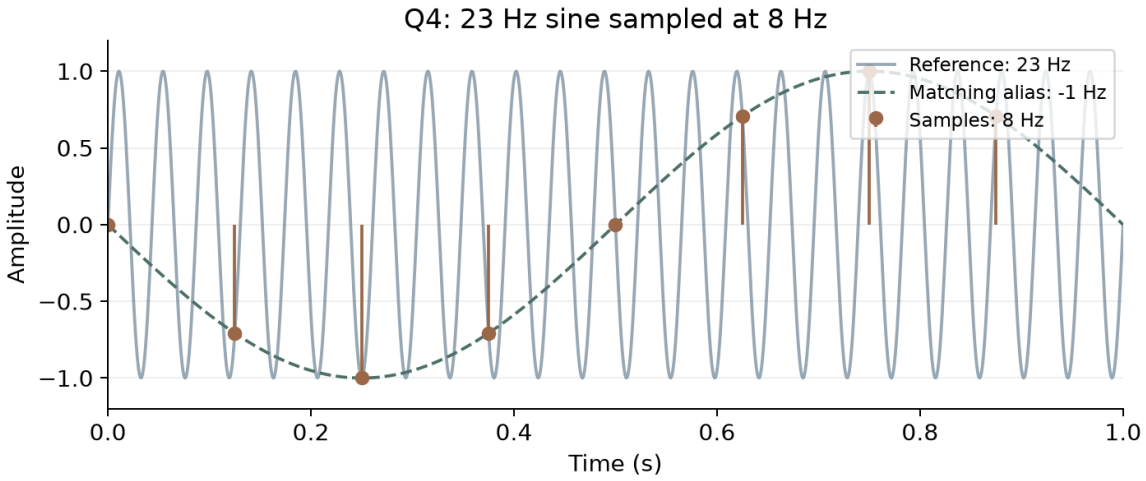


Figure 8: A 23 Hz signal sampled at 8 Hz produces a slow, reversed 1 Hz alias.

Q5. Describe a real-world scenario where understanding aliasing is crucial.

Answer: A camera can make spinning fan blades look slower or move backwards when the frame rate is too low. Choosing a high enough frame rate helps show the motion more accurately.

3.4 Image Sampling and Quantization

Q1. How does the resampling factor affect the image quality? At what factor does the distortion become apparent?

Answer: I tested resampling factors 2, 4 and 8. Factor 2 kept most details, while factor 4 made the image clearly softer and more blocky. At factor 8, the loss of detail was strong, so the distortion was easy to see.

Q2. What is the characteristic of the distortion introduced by resampling? How does it differ from the distortion introduced by requantization?

Answer: Resampling removes spatial detail and can make edges look blocky or jagged. Requantization keeps the same image size but reduces the number of gray levels, so smooth areas show banding or contour steps. The two distortions are different: one removes detail, and the other removes tone levels.

Q3. How few bits per pixel can be used in requantization before the image distortion becomes apparent?

Answer: I compared 6, 4 and 2 bits per pixel. At 6 bpp, the image looked close to the original. Banding became visible at 4 bpp and was strong at 2 bpp.



Figure 9: Grayscale Lenna: original, simple resampling (factor 4), and requantization (4 bpp).

Q4. Implement a function that applies both resampling and requantization to the image. Experiment with different combinations of factors and bits per pixel. What combination gives the best balance between compression and visual quality?

Answer: I tested four combinations. I preferred factor 2 with 4 bpp because the image still kept its main details. If the reduced pixels were stored at 4 bits each, they would need one-eighth of the original data. The images were enlarged again for comparison, so they are not compressed files.



Figure 10: Combined resampling and requantization of Lenna at the four requested settings.

Q5. Modify the code to implement a more advanced resampling method, such as bicubic interpolation. Compare the results with the simple resampling method used initially. Discuss the trade-offs between image quality and computational complexity.

Answer: I compared simple and bicubic resampling at factor 4. Bicubic gave smoother edges and fewer visible blocks. It uses more calculations than the simple method, so it is generally slower.

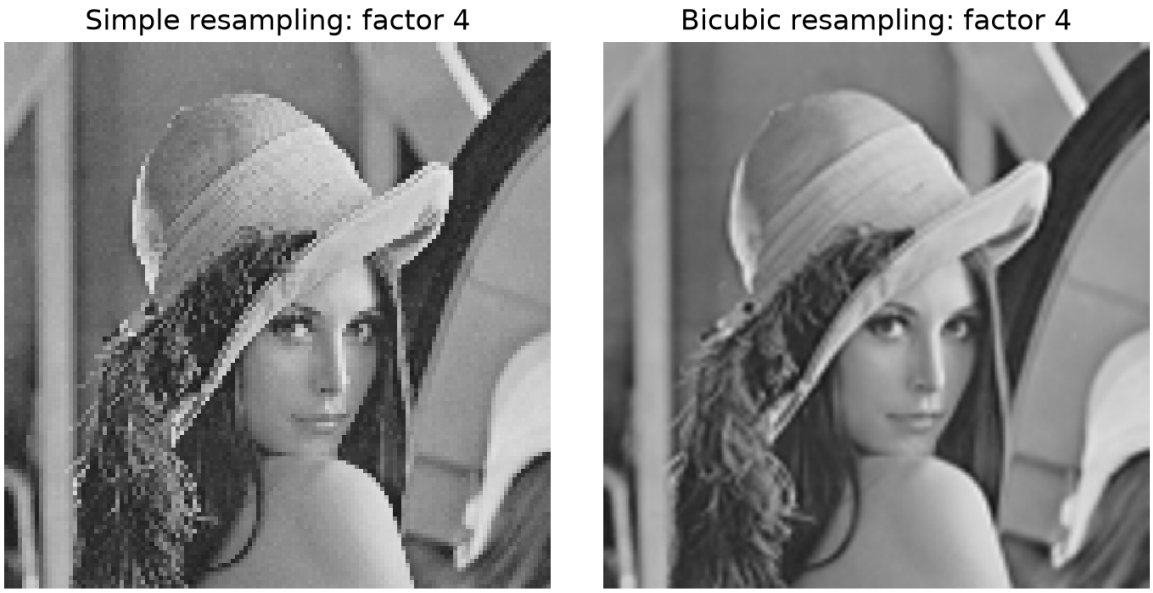


Figure 11: Simple and bicubic resampling of Lenna at factor 4.

Q6. Try using different images by changing the image_url. You can find a variety of test images at the USC-SIPI Image Database (`http://sipi.usc.edu/database/`). How do the effects of resampling and requantization vary for different types of images (e.g., portraits, textures, aerial photos)?

Answer: The portrait image kept its main shapes for longer, especially under requantization. The texture image lost fine detail sooner when I resampled it. Smooth regions showed banding more clearly, while detailed regions mainly lost texture.

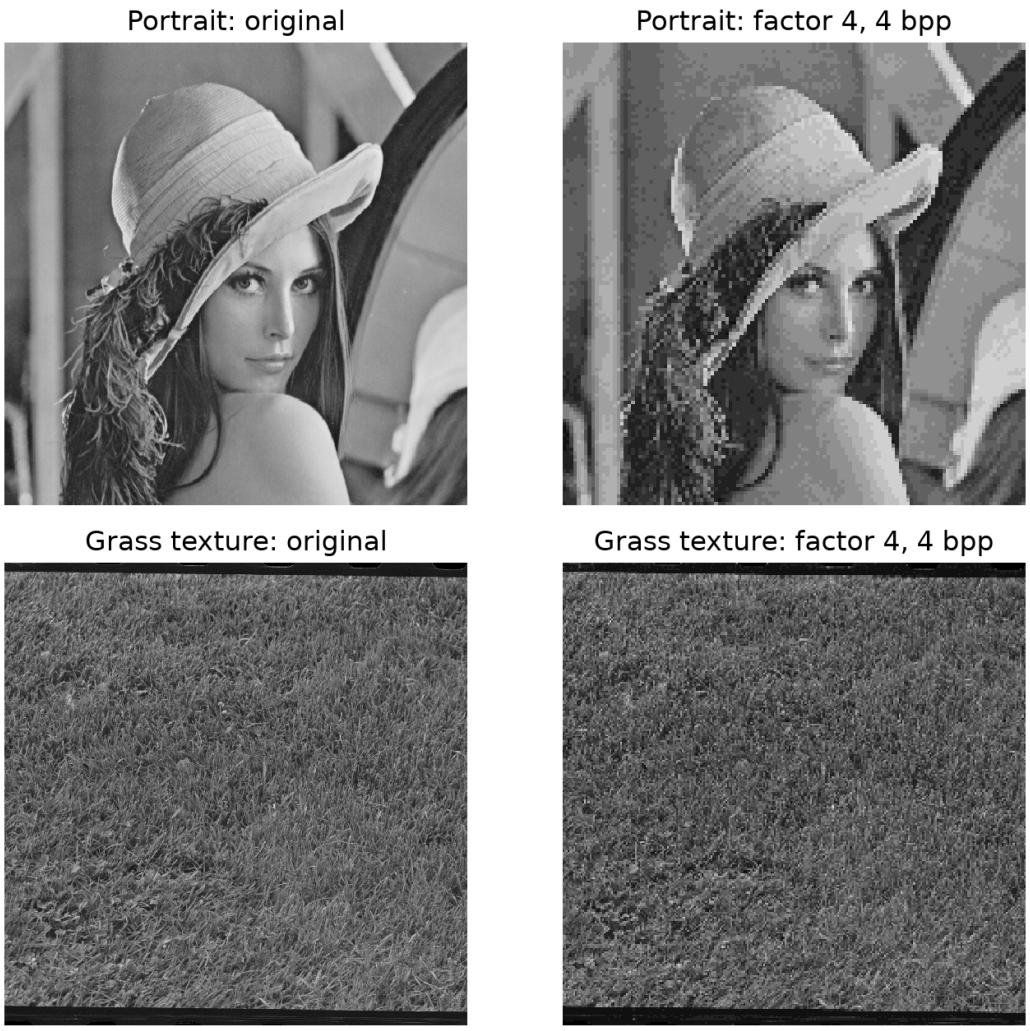


Figure 12: Portrait and grass texture before and after factor 4 resampling and 4 bpp requantization.

3.5 PSNR Calculation

Q1. Which process (resampling or requantization) introduces more distortion according to the PSNR values?

Answer: I compared the images using PSNR. Simple resampling at factor 4 had a lower PSNR than requantization at 4 bpp. This means resampling caused more pixel error in this test (Table 1).

Q2. How well do the PSNR values correlate with your visual perception of image quality?

Answer: The PSNR results mostly matched what I saw in the images. Resampling made the edges blocky, while requantization mainly caused banding. PSNR measures pixel error, but it cannot fully describe how an image looks to a person.

Q3. Create a function that calculates PSNR for different resampling factors and bits per pixel. Plot these results and discuss the trade-offs between compression and quality.

Answer: I tested different resampling factors and bit depths. Increasing the resampling factor generally lowered PSNR because more detail was lost. Using more bits per pixel improved PSNR because more gray levels were kept (Figure 13).

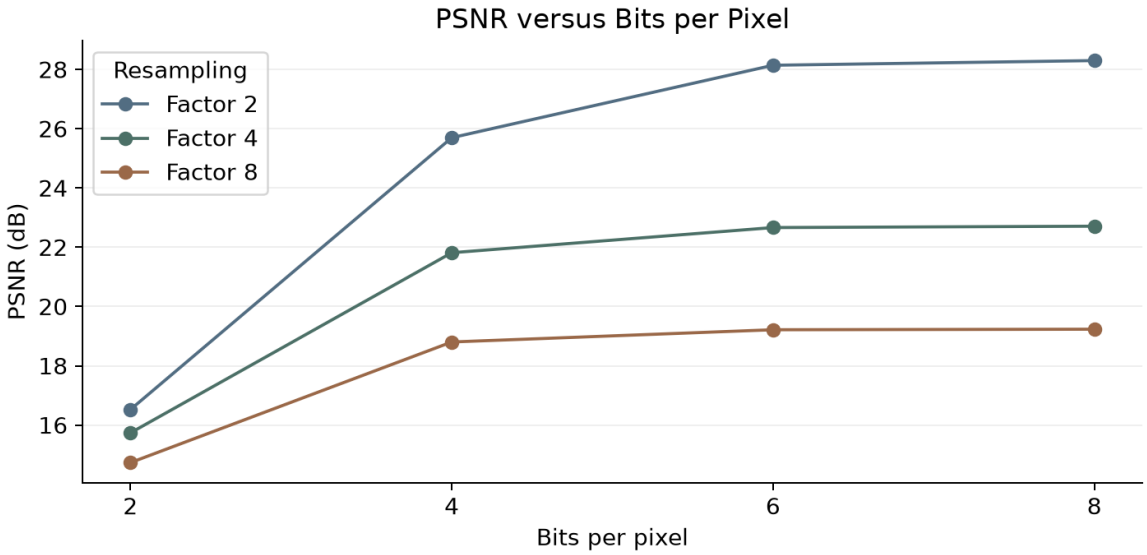


Figure 13: PSNR versus bits per pixel for resampling factors 2, 4 and 8.

Q4. Research the Structural Similarity Index (SSIM) as an alternative to PSNR. How does SSIM address some of the limitations of PSNR?

Answer: PSNR measures the average error between pixel values. SSIM also compares local brightness, contrast and structure. This helps describe image changes that PSNR may not show clearly.

Reference: Wang et al. (2004), *Image Quality Assessment: From Error Visibility to Structural Similarity.* DOI: 10.1109/TIP.2003.819861.

Q5. Implement the SSIM metric and compare its results with PSNR for your processed images. Discuss the differences in how these metrics evaluate image quality.

Answer: I calculated PSNR and SSIM for the same processed images (Table 1). A higher PSNR means less pixel error, while a higher SSIM means the image is more similar in local structure. The two scores measure image quality in different ways.

Table 1: PSNR and SSIM for the processed Lenna images.

Processing	PSNR (dB)	SSIM
Simple resampling, factor 4	22.7098	0.6589
Requantization, 4 bpp	29.2074	0.8832
Combined factor 2, 4 bpp	25.6964	0.7524
Bicubic resampling, factor 4	28.8441	0.8282

3.6 Noise Addition and PSNR Analysis

Q1. At what PSNR value does the noise start to become visibly apparent in the image?

Answer: I added different amounts of Gaussian noise and measured the PSNR. Slight noise was visible at $\sigma = 5$, with a PSNR of 34.0821 dB, especially in smooth parts of the image (Figures 14–15).

Q2. How does this PSNR threshold compare to the often-cited 30–50 dB range for acceptable image quality?

Answer: The 30–50 dB range is only a general guide. Noise can still be visible within this range. The image content also affects how easily we notice it.

Q3. At what noise level (standard deviation) does the image become unrecognizable? What is the corresponding PSNR?

Answer: I increased the noise until the image became difficult to recognize. At $\sigma = 500$, the PSNR was 6.2490 dB. Most details were lost, although some shapes could still be seen (Figure 15).

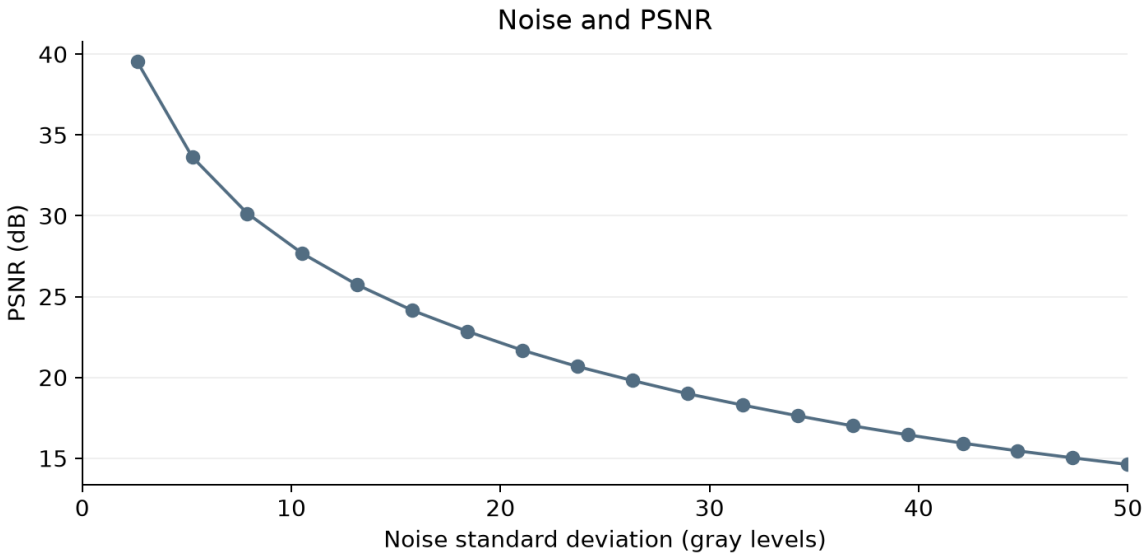


Figure 14: PSNR for 20 noise levels from 0 to 50. At $\sigma = 0$, PSNR is infinite, so that point is omitted from the curve.

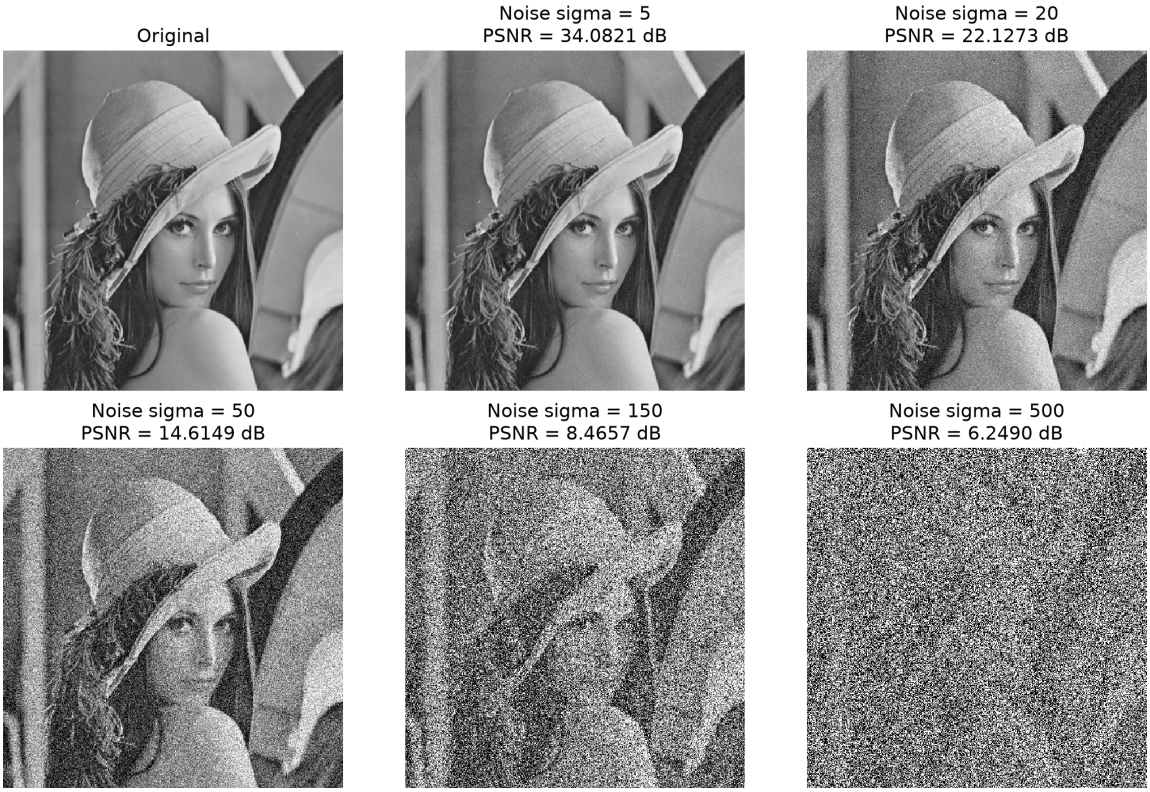


Figure 15: Original Lenna and Gaussian noise at standard deviations 5, 20, 50, 150 and 500. Each noisy image's title shows its measured PSNR.

Q4. Implement a simple noise reduction technique, such as a median filter or Gaussian blur. Apply this to the noisy images and compare the PSNR values before and after noise reduction. Discuss the effectiveness and limitations of this simple denoising approach.

Answer: I used a Gaussian filter on the image with noise $\sigma = 25$. The PSNR changed from 20.2170 dB to 29.0338 dB. The filter reduced noise, but it also made some edges and small details softer (Figure 16).

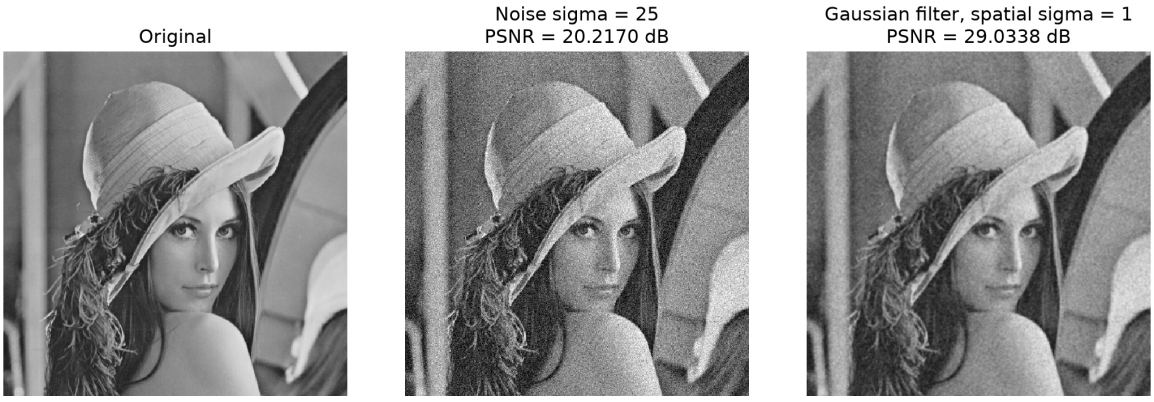


Figure 16: Original, noisy ($\sigma = 25$), and Gaussian filtered (spatial $\sigma = 1$) images. PSNR is measured against the original.

Q5. Research and briefly describe an advanced noise reduction technique. How does it improve upon simpler methods?

Answer: Non-Local Means compares small image patches and averages similar ones. It can reduce noise while keeping more edges and texture than simple blurring. It needs more calculations, so it is usually slower.

Reference: scikit-image documentation, `denoise_nl_means`: `https://scikit-image.org/docs/stable/api/skimage.restoration.html`.

3.7 Audio Signal Analysis

Q1. What is the duration of the audio clip? What is its highest frequency component?

Answer: I analyzed the piano recording. It lasted 0.5018 seconds, and its strongest FFT peak was at 219.2146 Hz. The highest frequency above my 1% threshold was 4832.6856 Hz. This is an estimate because weaker frequencies may still be present (Figure 17).

Q2. Describe the main features you observe in the spectrogram. What do the bright areas represent?

Answer: The piano sound has clear frequency bands that become weaker over time. The brighter areas show where the sound has more energy. This makes it easier to see how the sound changes (Figure 17).

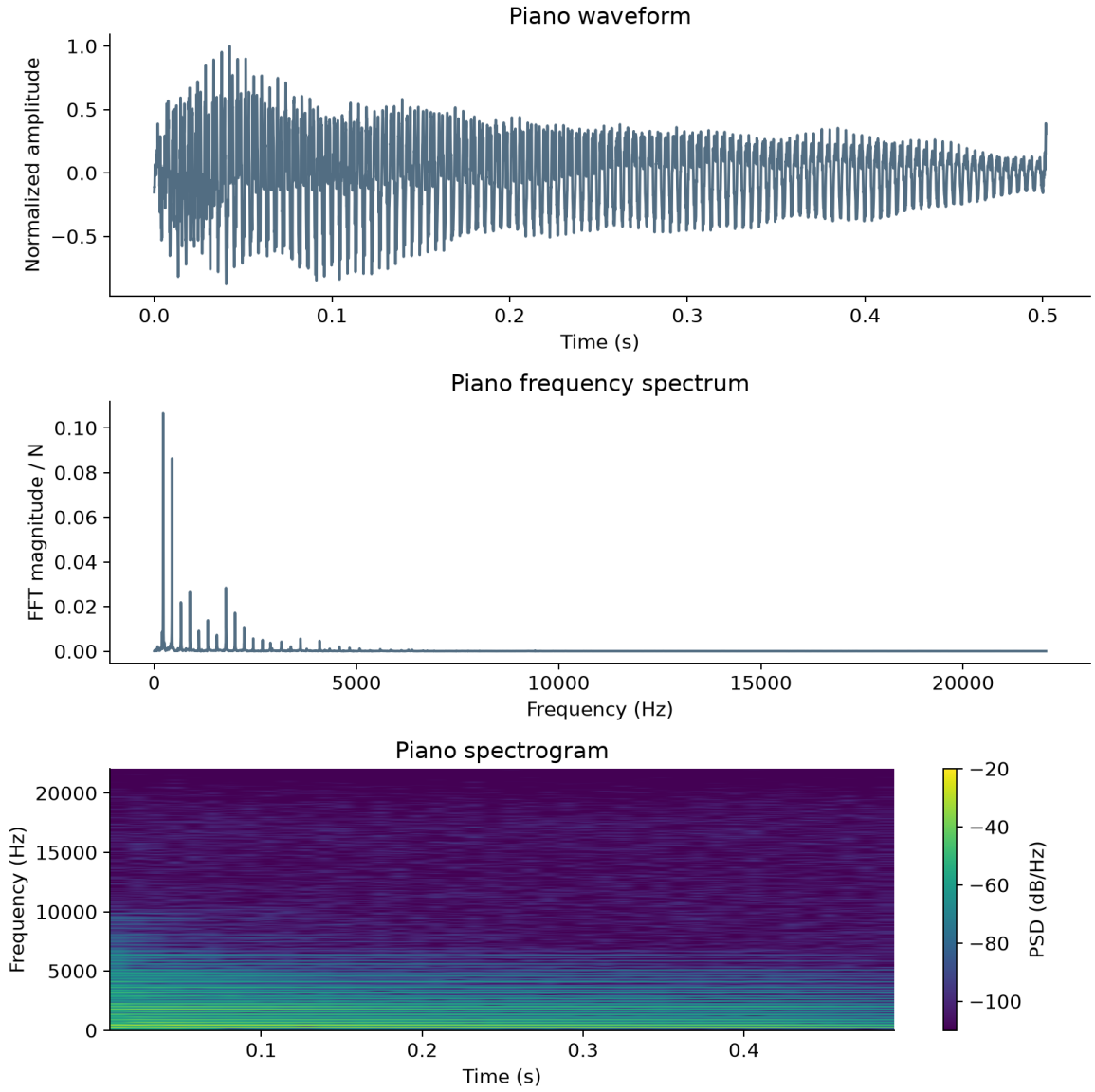


Figure 17: Piano waveform, normalized FFT magnitude and spectrogram.

Q3. Modify the code to analyze only the first half second of the audio. How does this change the frequency spectrum?

Answer: I compared the full 0.5018-second piano recording with its first 0.5 seconds. The spectra were almost the same because only 0.0018 seconds were removed. The main peak stayed near 220 Hz (Figure 18).

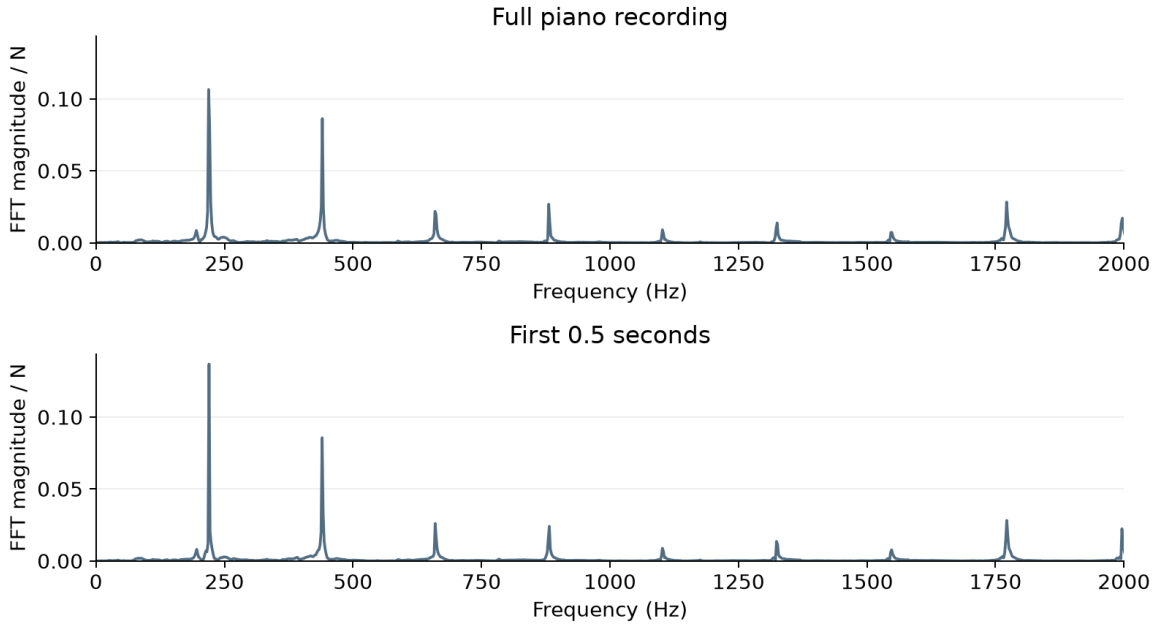


Figure 18: Full piano FFT and the first 0.5 seconds, shown over 0–2000 Hz with the same scales.

Q4. Try using different audio files by changing the audio_url. You can find a variety of sound effects and audio samples at `https://www.pacdv.com/sounds/`. How do the waveform, frequency spectrum, and spectrogram differ for different types of audio (e.g., music, speech, environmental sounds)?

Answer: I compared piano music with a thunderstorm recording. The piano had clearer repeating patterns and frequency peaks. The thunderstorm was more irregular, with sound spread over more frequencies. The spectrograms showed these differences over time (Figure 19).

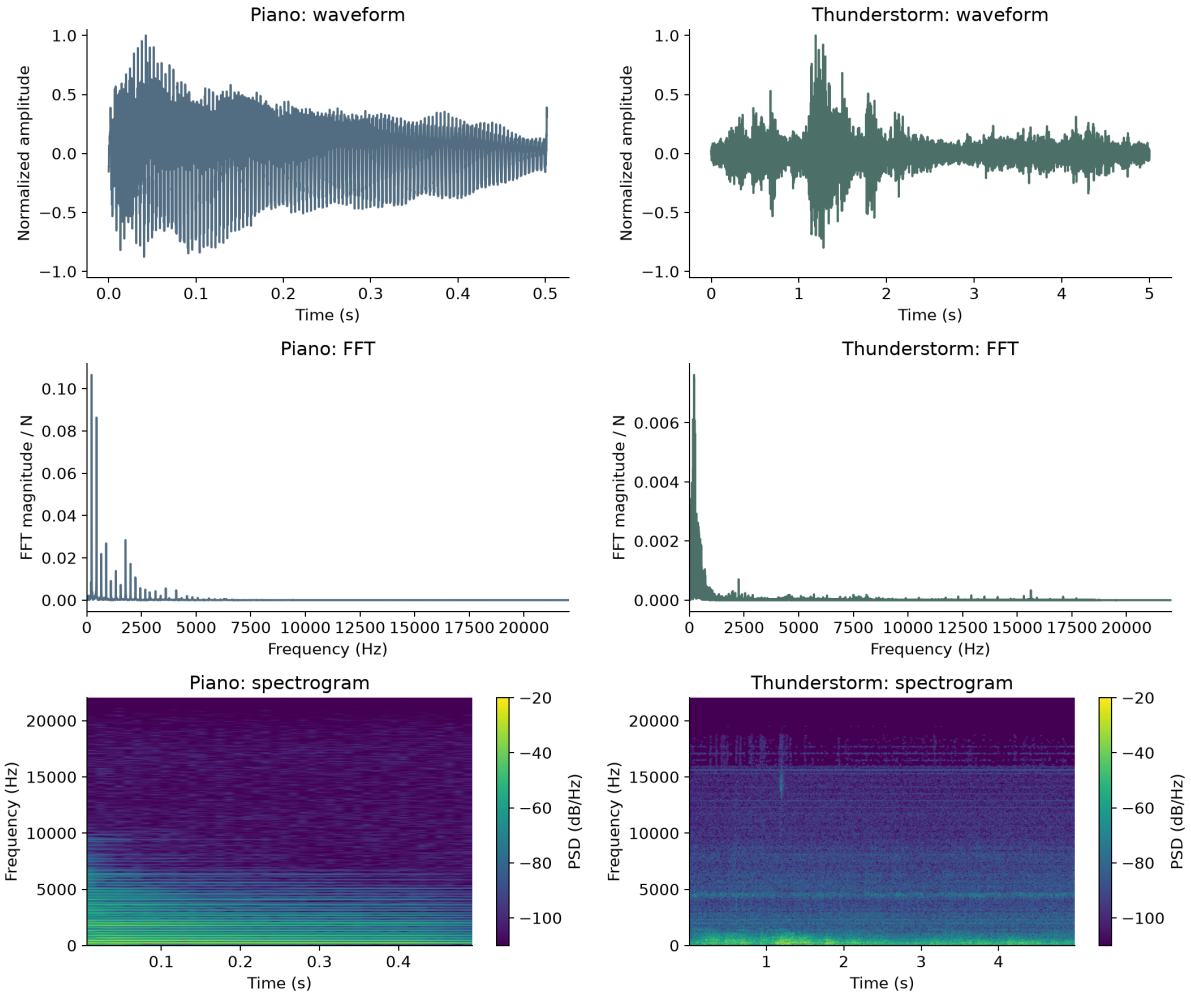


Figure 19: Piano and thunderstorm waveforms, FFTs and spectrograms. Each recording is normalized to its own peak; spectrograms use the same colour scale.

Q5. Implement a function to downsample the audio (reduce the sampling rate). How does this affect the audio quality and the frequency spectrum? At what point does aliasing become noticeable?

Answer: I reduced the piano sampling rate by factors 2, 4 and 8. At factor 8, the Nyquist limit dropped to 2756.25 Hz, so higher piano frequencies could cause aliasing. I also tested a 1800 Hz tone. Aliasing starts below 3600 Hz sampling. At 3000 Hz, the tone appeared at 1200 Hz, and at 2000 Hz it appeared at 200 Hz (Figure 20).

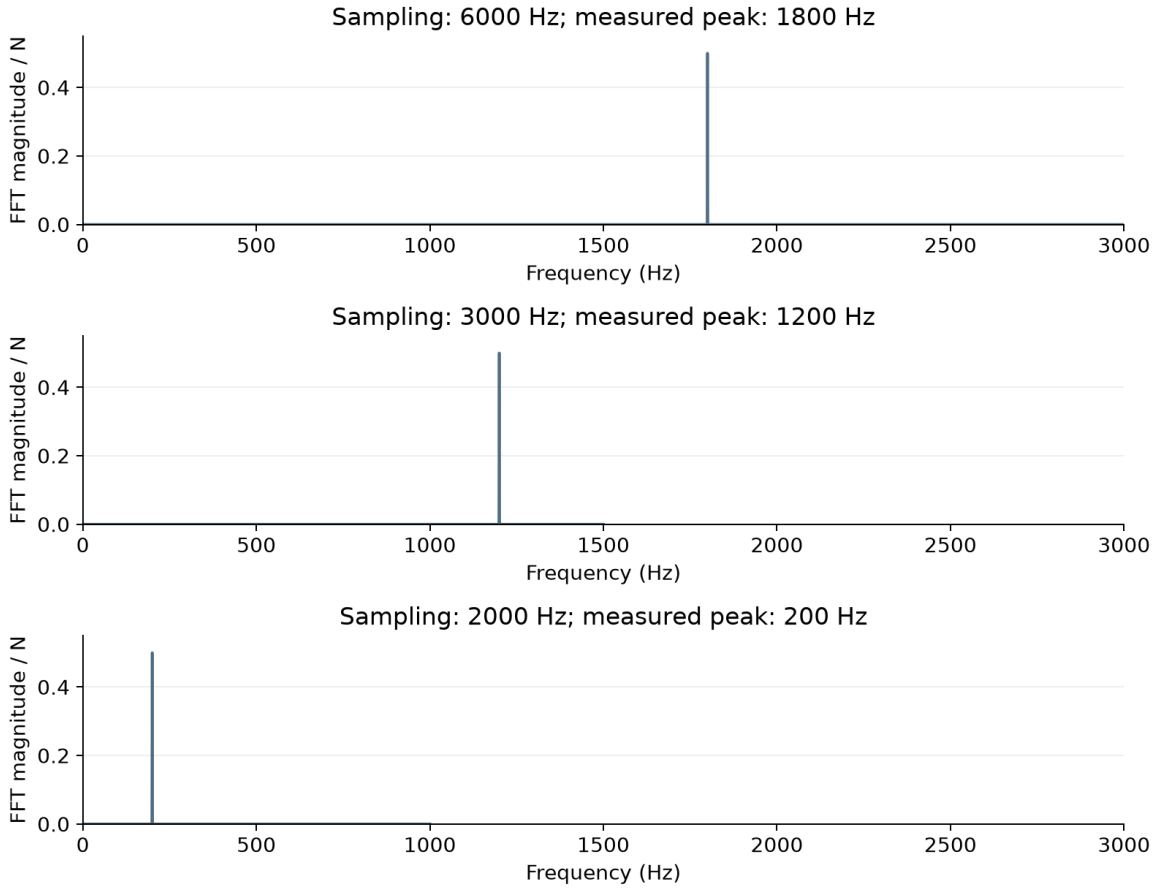


Figure 20: The 1,800 Hz test tone sampled at 6,000, 3,000 and 2,000 Hz gives FFT peaks at 1,800, 1,200 and 200 Hz.

Q6. Research and briefly describe a common audio processing technique (e.g., noise reduction, pitch shifting, time stretching). How might this technique be implemented using the concepts you've learned in this lab?

Answer: Pitch shifting changes how high or low a sound is. Playing audio samples at a different rate can change the pitch, but it also changes the duration. More advanced methods can change pitch while keeping the same length.

Reference: librosa effects documentation, `https://librosa.org/doc/0.11.0/effects.html`.

Audio sources: teacher's piano `a1.wav` and ESC-50 thunderstorm `1-101296-A-19.wav`.

4 Discussion

Resampling made images blocky, while requantization created banding. In the Lenna test, requantization at 4 bpp gave 29.2074 dB PSNR, compared with 22.7098 dB for factor-4 simple resampling. Adding Gaussian noise lowered PSNR. With noise $\sigma = 25$, Gaussian filtering increased it from 20.2170 to 29.0338 dB, but it also softened some details.

The sampling tests showed that low sampling rates can change the apparent frequency. The 1800 Hz tone appeared at 1200 Hz and 200 Hz when sampled too slowly. The piano clip was only 0.5018 seconds long, so comparing it with its first 0.5 seconds gave almost the same FFT. These results show why the processing settings matter.

5 Conclusion

The requested signals were generated and compared. The 20 Hz sine had twice as many cycles, and the composite had both curves and jumps. Fourier analysis identified the added 30 Hz component and the chirp's wider frequency range. The sampling experiments showed how aliasing can give a false impression of the original signal. The image comparisons showed different losses of detail and tone levels. Requantization at 4 bpp had less pixel error than simple resampling at factor 4. The noise tests showed how added noise affects PSNR. The 1,800 Hz tone appeared at lower frequencies when sampling was too slow.

These experiments showed me that sampling and quantization matter in signals, images and audio. Sampling controls how many values are recorded over time or across an image. Quantization limits the possible values, such as gray levels in an image or amplitude levels in audio. Images and audio can both use filtering and Fourier analysis, but images contain spatial patterns while audio changes over time. These methods can help reduce noise in medical images, improve sound recordings and prevent aliasing in communication systems.

Appendix: Code Snippets

```
sawtooth_wave = 2 * (5 * t - np.floor(5 * t)) - 1
square_wave_ceil = np.ceil(np.sin(2 * np.pi * 5 * t)) * 2 - 1
```

The first line removes whole cycles to make a repeating ramp. The second line is the teacher's replacement expression checked in Q3.

Example 1 – PSNR Calculation

```
def calculate_psnr(original, processed):
    error = original.astype(float) - processed.astype(float)
    mse = np.mean(error**2)
    if mse == 0:
        return float("inf")
    return 10 * np.log10(255**2 / mse)
```

I converted the pixel values to floats before subtraction to avoid calculation errors. PSNR measures the difference from the original image.

Example 2 – Gaussian Noise Reduction

```
noisy_25 = add_noise(original, 25)
filtered = gaussian_filter(noisy_25.astype(float), sigma=1)
noisy_psnr = calculate_psnr(original, noisy_25)
filtered_psnr = calculate_psnr(original, filtered)
```

I applied a Gaussian filter and compared PSNR before and after filtering.

Example 3 – Audio Downsampling

```
def downsample(data, sample_rate, factor):
    return data[::factor], sample_rate / factor
```

I kept every factor-th sample and reduced the sampling rate by the same factor. This method does not prevent aliasing.

Full Python scripts: `code/section1_1.py`, `code/section1_2.py`, `code/section1_3.py`, `code/section2_1.py` `code/section2_2.py`, `code/section2_3.py`, and `code/section3_1.py`.

Sources: DT081A Lab 1 assignment, Sections 1.1–3.1 (pages 1–5), and the DT081A lab report template.